LAPORAN PRAKTIKUM BIG DATA 2

Pemrosesan Terstruktur Menggunakan PySpark DataFrame API & Spark SQL

Nama Praktikan: Luthfi Akhyar Hasibuan (NIM: 52240014)

Dosen: ANDI PUJO RAHADI, Dr., S.T., M. Sc.

Mata Kuliah: Pemrograman Big Data

Dataset: NYC Yellow Taxi Trips (Jan 2021) - 1,369,765 records

---

Laporan ini merupakan dokumentasi komprehensif atas pengerjaan modul praktikum Big Data yang berfokus pada teknik ingestasi, transformasi waktu, agregasi berbasis DataFrame API, kueri Spark SQL, serta *Window Functions* untuk analitik tingkat lanjut.

1. Persiapan Data & Ingestasi (Explicit Schema)

Proses pembacaan awal dilakukan dengan mendefinisikan skema secara eksplisit (`StructType`) untuk memastikan integritas tipe data dan mengoptimalkan memori, menggantikan metode `inferSchema` yang kurang efisien untuk data bervolume jutaan baris.

```
from pyspark.sql import SparkSession from pyspark.sql import functions as F from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType # Definisi Skema Eksplisit schema = StructType([ StructField("vendor_id", StringType(), True), StructField("pickup_datetime", StringType(), True), StructField("dropoff_datetime", StringType(), True), StructField("passenger_count", IntegerType(), True), StructField("trip_distance", DoubleType(), True), # ... (Kolom lainnya disesuaikan dengan skema CSV) StructField("total_amount", DoubleType(), True) ]) df_raw = spark.read.format("csv") \ .option("header", "true") \ .schema(schema) \ .load("/dbfs/FileStore/tables/nyc_taxi.csv") print(f"Total Record Data: {df_raw.count():,}")
```

```
Total Record Data: 1,369,765
```

Ringkasan Eksekusi: Data berhasil dimuat dengan sempurna tanpa pergeseran nilai kolom. Dataset berisi **1,36 juta baris** transaksi perjalanan taksi yang siap diproses lebih lanjut.

2. Pembersihan Data & Feature Engineering

Data yang kosong (null) dan bernilai tidak wajar (jarak/tarif = 0) dibuang. Dilakukan ekstraksi kolom waktu (`pickup_hour`) dan kalkulasi durasi perjalanan dalam menit (`trip_duration_min`).

```
# Cleaning & Datetime Extraction df_clean = df_raw.dropna(subset=
["pickup_datetime", "total_amount"]) \ .filter((F.col("trip_distance") > 0) &
(F.col("total_amount") > 0)) df_transformed = df_clean \ .withColumn("pickup_ts",
F.to_timestamp("pickup_datetime", "yyyy-MM-dd HH:mm:ss")) \
.withColumn("dropoff_ts", F.to_timestamp("dropoff_datetime", "yyyy-MM-dd
HH:mm:ss")) \ .withColumn("pickup_hour", F.hour("pickup_ts")) \
.withColumn("trip_duration_min", F.round((F.unix_timestamp("dropoff_ts") -
F.unix_timestamp("pickup_ts")) / 60, 2))
df_transformed.createOrReplaceTempView("v_nyc_taxi")
df_transformed.select("pickup_hour", "trip_distance", "trip_duration_min",
"total_amount").show(3)
```

pickup_hour	trip_distance	trip_duration_min	total_amount
0	2.1	6.03	11.8
0	0.2	0.98	4.3
0	14.7	27.6	51.95

3. Analisis Kategori A (DataFrame API)

Tugas A1: Analisis Jam Sibuk Penumpang (Peak Hours)

```
# Agregasi Total Frekuensi, Rata-rata Jarak, dan Total Pendapatan per Jam
df_tugas_a1 = df_transformed.groupBy("pickup_hour") \ .agg(
F.count("*").alias("total_frekuensi"), F.round(F.avg("trip_distance"),
2).alias("rata_rata_jarak"), F.round(F.sum("total_amount"),
2).alias("total_pendapatan") ) \ .orderBy(F.desc("total_frekuensi"))
df_tugas_a1.show(5)
```

pickup_hour	total_frekuensi	rata_rata_jarak	total_pendapatan
15	111152	3.23	1,900,495.60
14	110446	4.75	1,873,726.94
16	104812	2.83	1,888,702.54

pickup_hour	total_frekuensi	rata_rata_jarak	total_pendapatan
17	102850	2.93	1,867,686.27
13	102377	5.68	1,685,057.87

Analisis Hasil: Jam operasional tersibuk berada pada rentang sore hari, di mana puncak pemesanan tertinggi terjadi pada pukul **15:00 (3 Sore)** dengan 111 ribu lebih perjalanan, menghasilkan pendapatan total menembus angka $1,9 juta.

Tugas A2: Durasi Perjalanan Berdasarkan Jumlah Penumpang

```
df_tugas_a2 = df_transformed.filter(F.col("passenger_count") > 0) \
.groupBy("passenger_count") \ .agg( F.round(F.avg("trip_duration_min"),
2).alias("rata_rata_durasi_menit"), F.round(F.avg("total_amount"),
2).alias("rata_rata_total_tarif") ) \ .orderBy("passenger_count")
df_tugas_a2.show()
```

passenger_count	rata_rata_durasi_menit	rata_rata_total_tarif
1	12.78	16.36
2	14.19	17.05
3	15.39	16.81
4	15.30	17.22
5	21.31	16.55
6	15.50	16.22
7	25.00	101.71
8	19.85	16.30

Analisis Hasil: Rata-rata tarif untuk rombongan reguler (1-6 penumpang) sangat stabil di kisaran $16-$17. Terdeteksi sebuah pencilan data (anomali) pada kelompok 7 penumpang dengan lonjakan tarif rata-rata mencapai $101.71, yang mengindikasikan jenis pesanan khusus atau taksi limusin.

4. Analisis Kategori B (Spark SQL)

Tugas B1: Segmentasi Jarak Tempuh (CASE WHEN)

```
df_tugas_b1 = spark.sql(""" SELECT CASE WHEN trip_distance <= 2.0 THEN 'Short Trip' WHEN trip_distance > 2.0 AND trip_distance <= 10.0 THEN 'Medium Trip' ELSE 'Long Trip' END AS trip_segment, COUNT(*) AS total_trips, ROUND(AVG(fare_amount), 2) AS avg_fare, ROUND(AVG(tip_amount), 2) AS avg_tip FROM v_nyc_taxi GROUP BY 1 ORDER BY total_trips DESC """)
```

trip_segment	total_trips	avg_fare	avg_tip
Short Trip	779134	6.77	1.44
Medium Trip	492818	15.52	2.38
Long Trip	71692	46.22	4.30

Analisis Hasil: Layanan taksi di NYC sangat didominasi oleh mobilitas jarak dekat (Short Trip) yang mencapai lebih dari 779 ribu perjalanan. Semakin jauh jarak yang ditempuh (Long Trip), besaran tip rata-rata (avg_tip) yang diberikan oleh pelanggan juga meningkat secara proporsional.

Tugas B2: Analisis Metode Pembayaran & Tip Rate

```
df_tugas_b2 = spark.sql(""" SELECT payment_type, ROUND(AVG((tip_amount / total_amount) * 100), 2) AS avg_tip_percentage FROM v_nyc_taxi GROUP BY payment_type ORDER BY avg_tip_percentage DESC """)
```

payment_type	avg_tip_percentage (%)
1 (Credit Card)	15.47
NULL	6.52
3	0.01
4	0.0
2 (Cash)	0.0

Analisis Hasil: Pembayaran menggunakan Kartu Kredit (Type 1) merekam persentase tip tertinggi (15,47%). Sebaliknya, pembayaran Tunai (Type 2) menghasilkan tip 0.0%, bukan karena penumpang pelit, melainkan karena sopir mengantongi tip tunai secara langsung tanpa dimasukkan ke sistem argometer.

5. Kategori C (Advanced Functions & Export)

Tugas C1: Ranking Perjalanan Terjauh Per Jam (Window Function)

```
from pyspark.sql.window import Window window_spec = Window.partitionBy("pickup_hour").orderBy(F.desc("trip_distance")) df_tugas_c1 = df_transformed \ .withColumn("jarak_rank", F.dense_rank().over(window_spec)) \ .filter(F.col("jarak_rank") == 1) \ .filter(F.col("pickup_hour").isin(8, 12, 18)) \ .select("pickup_hour", "trip_distance", "total_amount", "jarak_rank") \ .orderBy("pickup_hour")
```

pickup_hour	trip_distance (miles)	total_amount ($)	jarak_rank
8	112257.10	17.66	1
12	51676.03	38.84	1
18	59.90	310.56	1

Analisis Hasil: Fungsi Window berhasil mengidentifikasi *Data Glitches* (kerusakan sensor argon/GPS). Jarak 112.257 mil pada jam 08:00 sangat tidak realistis, apalagi tarifnya hanya $17.66. Sebaliknya, jarak terjauh pada jam 18:00 (59,9 mil) dengan tarif $310.56 terlihat jauh lebih logis. Ini menyoroti pentingnya tahapan pembersihan lanjutan di masa depan.

Tugas C2: Export Hasil Agregasi ke Format Parquet

```
output_path = "/dbfs/FileStore/tables/output/nyc_hourly_summary.parquet" df_tugas_a1.write \ .mode("overwrite") \ .parquet(output_path) print(f"Data agregasi jam sibuk telah disimpan di: {output_path}")
```

```
--- TUGAS C2: EXPORT PARQUET BERHASIL --- Data agregasi jam sibuk telah berhasil disimpan di: /dbfs/FileStore/tables/output/nyc_hourly_summary.parquet Verifikasi: File/Folder Parquet ditemukan!
```

Kesimpulan Penutup: Proses rekayasa data diakhiri dengan mengekspor dataset yang sudah dibersihkan dan diagregasi ke dalam format *Parquet*. Format *columnar* ini dipilih karena kemampuannya dalam kompresi yang jauh lebih efisien, lebih hemat ruang (disk-space), dan lebih cepat dibaca untuk kueri Big Data (Athena/Presto/Spark) dibandingkan format CSV.

LAPORAN PRAKTIKUM PEMROGRAMAN BIG DATA

Pemrosesan Terstruktur Menggunakan PySpark DataFrame API & Spark SQL

Nama: Luthfi Akhyar Hasibuan (NIM: 52240014)

Dosen: ANDI PUJO RAHADI, Dr., S.T., M. Sc.

Mata Kuliah: Pemrograman Big Data

Dataset: NYC Yellow Taxi Trips Dataset (1,369,765 record)

Ringkasan Eksekutif: Laporan ini mendokumentasikan pemrosesan data terstruktur skala besar menggunakan **Spark DataFrame API** dan **Spark SQL**. Dataset yang digunakan adalah NYC Yellow Taxi Trip Data (Januari 2021), mencakup tahapan *data ingestion* dengan skema eksplisit, rekayasa fitur waktu, agregasi kompleks, segmentasi data dengan kueri bersyarat, hingga implementasi *window functions* untuk mendeteksi anomali perjalanan.

1. Pembersihan Data & Ekstraksi Fitur (DataFrame API)

Memfilter baris yang memiliki nilai null dan jarak nol, dilanjutkan dengan ekstraksi *timestamp* untuk mendapatkan durasi perjalanan (`trip_duration_min`) dan jam penjemputan (`pickup_hour`).

```
df_clean = df_raw.dropna(subset=["pickup_datetime", "total_amount"]) \
    .filter((F.col("trip_distance") > 0) & (F.col("total_amount") > 0))

df_transformed = df_clean \
    .withColumn("pickup_ts", F.to_timestamp("pickup_datetime", "yyyy-MM-dd
    .withColumn("dropoff_ts", F.to_timestamp("dropoff_datetime", "yyyy-MM-d
    .withColumn("pickup_hour", F.hour("pickup_ts")) \
    .withColumn("trip_duration_min", F.round((F.unix_timestamp("dropoff_ts"

# Mendaftarkan Temporary View
df_transformed.createOrReplaceTempView("v_nyc_taxi")
```

--- CONTOH DATA TERTATA (5 BARIS) --- +---------+-----------+-------------+-----------------+------------+ |vendor_id|pickup_hour|trip_distance|trip_duration_min|total_amount| +---------+-----------+-------------+-----------------+------------+ | 1| 0| 2.1| 6.03| 11.8| | 1| 0| 0.2| 0.98| 4.3| | 1| 0| 14.7| 27.6| 51.95| | 1| 0| 10.6| 15.22| 36.35| | 2| 0| 4.94| 16.53| 24.36| +---------+-----------+-------------+-----------------+------------+

2. Kategori A: Analisis Tren & Metrik (DataFrame API)

Tugas A1: Analisis Jam Sibuk Penumpang

```
df_tugas_a1 = df_transformed.groupBy("pickup_hour") \
    .agg(
        F.count("*").alias("total_frekuensi"),
        F.round(F.avg("trip_distance"), 2).alias("rata_rata_jarak"),
        F.round(F.sum("total_amount"), 2).alias("total_pendapatan")
    ) \
    .orderBy(F.desc("total_frekuensi"))
```

--- TUGAS A1: TOP 5 JAM SIBUK PENUMPANG --- +-----------+---------------+---------------+----------------+ |pickup_hour|total_frekuensi|rata_rata_jarak|total_pendapatan| +-----------+---------------+---------------+----------------+ | 15| 111152| 3.23| 1900495.6| | 14| 110446| 4.75| 1873726.94| | 16| 104812| 2.83| 1888702.54| | 17| 102850| 2.93| 1867686.27| | 13| 102377| 5.68| 1685057.87| +-----------+---------------+---------------+----------------+

Analisis: Jam paling sibuk untuk taksi NYC adalah sore hari, dengan puncaknya pada pukul 15:00 yang menangani lebih dari 111 ribu perjalanan dan mencatatkan pendapatan sekitar 1,9 juta USD.

Tugas A2: Durasi Perjalanan Berdasarkan Jumlah Penumpang

```
df_tugas_a2 = df_transformed.filter(F.col("passenger_count") > 0) \
    .groupBy("passenger_count") \
    .agg(
        F.round(F.avg("trip_duration_min"), 2).alias("rata_rata_durasi_meni
        F.round(F.avg("total_amount"), 2).alias("rata_rata_total_tarif")
    ) \
    .orderBy("passenger_count")
```

--- TUGAS A2: ANALISIS DURASI & TARIF BERDASARKAN PENUMPANG --- +---------------+---------------------+--------------------+ |passenger_count|rata_rata_durasi_menit|rata_rata_total_tarif| +---------------+---------------------+--------------------+ | 1| 12.78| 16.36| | 2| 14.19| 17.05| ... | 7| 25.0| 101.71| | 8| 19.85| 16.3| +---------------+---------------------+--------------------+

Analisis: Rata-rata tarif untuk kelompok 1 hingga 6 penumpang sangat stabil di kisaran $16 - $17. Terjadi lonjakan signifikan pada kelompok 7 penumpang ($101.71), yang mengindikasikan penyewaan khusus kendaraan besar untuk jarak jauh.

3. Kategori B: Analisis SQL & Segmentasi Tarif (Spark SQL)

Tugas B1: Segmentasi Jarak Tempuh Menggunakan CASE WHEN

```
df_tugas_b1 = spark.sql("""
    SELECT
        CASE
            WHEN trip_distance <= 2.0 THEN 'Short Trip'
            WHEN trip_distance > 2.0 AND trip_distance <= 10.0 THEN 'Medium
            ELSE 'Long Trip'
        END AS trip_segment,
        COUNT(*) AS total_trips,
        ROUND(AVG(fare_amount), 2) AS avg_fare,
        ROUND(AVG(tip_amount), 2) AS avg_tip
    FROM v_nyc_taxi
    GROUP BY 1 ORDER BY total_trips DESC
""")
```

--- TUGAS B1: SEGMENTASI JARAK TEMPUH --- +------------+-----------+--------+-------+ |trip_segment|total_trips|avg_fare|avg_tip| +------------+-----------+--------+-------+ | Short Trip| 779134| 6.77| 1.44| | Medium Trip| 492818| 15.52| 2.38| | Long Trip| 71692| 46.22| 4.3| +------------+-----------+--------+-------+

Analisis: Sebagian besar pengguna layanan taksi (779 ribu transaksi) memanfaatkan taksi untuk rute pendek (Short Trip). Walau demikian, rata-rata *tip* untuk perjalanan jauh (Long Trip) terbukti hampir 3 kali lipat lebih besar.

Tugas B2: Analisis Metode Pembayaran & Tip Rate

```
df_tugas_b2 = spark.sql("""
    SELECT
        payment_type,
        ROUND(AVG((tip_amount / total_amount) * 100), 2) AS avg_tip_percent
    FROM v_nyc_taxi
    GROUP BY payment_type
    ORDER BY avg_tip_percentage DESC
""")
```

--- TUGAS B2: ANALISIS METODE PEMBAYARAN & TIP RATE --- +------------+-----------------+ |payment_type|avg_tip_percentage| +------------+-----------------+ | 1| 15.47| | NULL| 6.52| | 3| 0.01| | 4| 0.0| | 2| 0.0| +------------+-----------------+

Analisis: Pembayaran elektronik dengan Kartu Kredit (Tipe 1) memiliki persentase tip rata-rata sebesar 15,47%. Sementara itu, tipe pembayaran Tunai/Cash (Tipe 2) tercatat 0%, selaras dengan praktik operasional di lapangan bahwa uang tip tunai umumnya tidak dicatat secara historis dalam argometer.

4. Kategori C: Advanced Window Functions & Export

Tugas C1: Ranking Perjalanan Terjauh Per Jam

```
window_spec = Window.partitionBy("pickup_hour").orderBy(F.desc("trip_distan

df_tugas_c1 = df_transformed \
    .withColumn("jarak_rank", F.dense_rank().over(window_spec)) \
    .filter(F.col("jarak_rank") == 1) \
    .filter(F.col("pickup_hour").isin(8, 12, 18)) \
    .select("pickup_hour", "trip_distance", "total_amount", "jarak_rank") \
    .orderBy("pickup_hour")
```

--- TUGAS C1: PERJALANAN TERJAUH DI JAM 08:00, 12:00, DAN 18:00 --- +-----------+-------------+-----------+----------+ |pickup_hour|trip_distance|total_amount|jarak_rank| +-----------+-------------+-----------+----------+ | 8| 112257.1| 17.66| 1| | 12| 51676.03| 38.84| 1| | 18| 59.9| 310.56| 1| +-----------+-------------+-----------+----------+

Analisis: Terdapat anomali data (*glitch error*) yang ekstrem di pukul 08:00 dan 12:00, mencatatkan jarak tak wajar ratusan ribu mil dengan tarif rendah. Data pukul 18:00 merupakan jarak terjauh yang wajar (59.9 mil) untuk biaya $310. Window Function ini krusial dalam Quality Control pipeline Big Data.

Tugas C2: Export Hasil Agregasi ke Parquet Format

```
output_path = "/dbfs/FileStore/tables/output/nyc_hourly_summary.parquet"

df_tugas_a1.write \
    .mode("overwrite") \
    .parquet(output_path)
```

--- TUGAS C2: EXPORT PARQUET BERHASIL --- Data agregasi jam sibuk telah berhasil disimpan di: /dbfs/FileStore/tables/output/nyc_hourly_summary.parquet Verifikasi: File/Folder Parquet ditemukan!